Mạng cho DevOps: DNS, TCP, HTTP, TLS và chẩn đoán

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Mạng: request đi từ máy bạn đến dịch vụ như thế nào?

IP là địa chỉ của thiết bị trong một mạng. Port giúp máy phân biệt dịch vụ nào nhận kết nối. DNS đổi tên miền thành địa chỉ IP. TCP giúp hai đầu trao đổi dữ liệu có thứ tự và báo lỗi truyền; HTTP là ngôn ngữ request/response ở tầng ứng dụng. TLS bảo vệ kênh truyền và xác thực máy chủ theo chứng chỉ. Một URL như **https://example.com:443/notes** chứa scheme, host, port và path. Port 443 là mặc định của HTTPS nên thường không hiện.

Khi bạn mở URL, máy cần tìm IP của tên miền, tạo kết nối tới IP:port, thỏa thuận TLS nếu dùng HTTPS, gửi HTTP request, rồi nhận response. Một lỗi có thể xảy ra ở từng bước. **NXDOMAIN** gợi ý vấn đề tên miền; timeout gợi ý đường mạng/firewall/dịch vụ không đáp ứng; **connection refused** gợi ý không có tiến trình lắng nghe hoặc bị từ chối ngay; HTTP 404 nghĩa là đã nhận phản hồi HTTP nhưng path không được tìm thấy; HTTP 500 gợi ý phía server xử lý thất bại. Đây là chỉ dẫn ban đầu, cần đối chiếu log và cấu hình.

Địa chỉ **127.0.0.1** hoặc **localhost** trỏ tới chính máy đang chạy lệnh. Nếu ứng dụng trong container hoặc VM, **localhost** ở đó là container/VM đó, không tự động là máy chủ hay máy của bạn. Rất nhiều lỗi kết nối database xuất phát từ hiểu nhầm phạm vi của **localhost**.

Lab quan sát một request

Điều kiện: có trình duyệt và terminal. Mở trang web công khai bạn tin tưởng. Trong browser, mở Developer Tools -> Network, tải lại trang, chọn một request. Ghi method, URL, status code và thời gian. Trên PowerShell, có thể dùng **Test-NetConnection example.com -Port 443**; trên Linux dùng **curl -I https://example.com**. Không gộp lỗi DNS, TCP và HTTP thành một "lỗi mạng"; ghi bước cụ thể thất bại.

Bài tập luồng request

Vẽ bằng chữ: browser -> DNS -> internet/router -> reverse proxy/load balancer -> ứng dụng -> database. Đánh dấu ở đâu có IP, port, TLS và HTTP. Nếu browser nhận HTTP 502, bạn sẽ xem proxy/upstream; nếu nhận HTTP 200 nhưng dữ liệu rỗng, bạn sẽ xem mã ứng dụng và truy vấn database trước.

Điều tra một lỗi mạng theo từng lớp

Khi người dùng nói "không vào được ứng dụng", một người mới thường thử đổi DNS, mở firewall hoặc restart server ngay. Cách có hệ thống bắt đầu từ việc xác định **điểm cuối đã chứng minh hoạt động**. Nếu **GET /health** từ máy chủ trả 200 nhưng từ máy người dùng timeout, ứng dụng có thể ổn và vấn đề nằm trên đường mạng. Nếu từ cả hai nơi đều trả HTTP 500, request đã đi tới ứng dụng và cần xem log hoặc dependency. Nếu trình duyệt báo lỗi chứng chỉ, TLS là lớp cần kiểm tra trước khi đụng database.

Hãy chia chuỗi thành sáu câu hỏi: tên miền phân giải thành IP nào; có đi được đến IP/port không; có hoàn tất TLS không; HTTP request có tới proxy không; proxy có chuyển đến ứng dụng không; ứng dụng có đọc được database không. Mỗi câu có một loại bằng chứng. DNS dùng lookup; TCP dùng kiểm tra

kết nối; TLS dùng thông tin certificate và handshake; HTTP dùng status/header; ứng dụng dùng log; database dùng log/kết nối. Không một lệnh nào chứng minh toàn bộ chuỗi.

Lab với server local

Điều kiện: Python và một terminal. Tạo thư mục lab trống, mở terminal ở đó, chạy **python -m http.server 8001** (Windows có thể dùng **py -m http.server 8001**). Mở **http://127.0.0.1:8001** trong trình duyệt; mong đợi trang liệt kê thư mục. Khi server đang chạy, thử **http://127.0.0.1:8002**; dự đoán lỗi. Quay về terminal, quan sát request log. Dừng server bằng Ctrl+C rồi mở lại URL 8001; dự đoán lỗi mới. Bài này chứng minh vai trò của process và port mà chưa cần cloud.

Thử thêm **http://localhost:8001** khi server chạy. Trên nhiều máy nó tới cùng dịch vụ qua tên loopback, nhưng cách phân giải IPv4/IPv6 có thể khác. Nếu hai URL không cho cùng kết quả, đây là cơ hội quan sát DNS/hosts và địa chỉ server đang bind, không phải lỗi cần che bằng một cấu hình đoán mò.

Tình huống có lời giải thích

Bạn thấy trình duyệt nhận 404 từ Notes API ở **/unknown**. Điều gì đã hoạt động? Ít nhất client tới được IP/port và tiến trình đã trả HTTP response. Lỗi có thể là path không tồn tại. Bạn thấy timeout không có log trên server: request có thể chưa tới ứng dụng; kiểm tra client, cổng, firewall hoặc proxy. Bạn thấy 500 và log nêu SQLite không mở được file: kiểm tra đường dẫn và quyền file database. Ba tình huống giống nhau ở cảm giác "ứng dụng hỏng" nhưng dẫn đến ba nhánh điều tra khác.

Đọc một thiết kế network cloud

Giả sử mạng ảo dùng dải **10.20.0.0/16**, trong đó subnet public cho load balancer và subnet private cho ứng dụng/database. Ký hiệu **/16** cho biết số bit đầu cố định của địa chỉ; subnet như **10.20.1.0/24** và **10.20.2.0/24** là các phần nhỏ hơn. Bạn chưa cần tính nhẩm mọi địa chỉ, nhưng phải hiểu rằng hai subnet có phạm vi địa chỉ và route khác nhau. "Public" không chỉ do tên subnet: cần xét route tới internet, public IP, bộ lọc mạng và dịch vụ có đang lắng nghe hay không.

Khi người dùng truy cập **https://notes.example**, DNS trỏ tới điểm vào. TLS có thể kết thúc ở load balancer hoặc proxy theo thiết kế. Điểm vào chuyển tới API trong subnet private; API nói chuyện database qua đường private. Nếu API lỗi kết nối DB, theo lớp: host/port cấu hình; DNS nội bộ; route và firewall; DB đang chạy; xác thực; giới hạn kết nối. Không mở port DB cho **0.0.0.0/0** chỉ để thử; nó làm tăng phạm vi tấn công mà không chứng minh nguyên nhân.

Bài tập CIDR và luồng traffic

Vẽ hai subnet **10.20.1.0/24** và **10.20.2.0/24**, đặt API ở subnet thứ nhất, DB ở subnet thứ hai. Ghi rule ý tưởng: chỉ API được kết nối tới cổng DB; người dùng chỉ tới cổng HTTPS của điểm vào. Sau đó nêu ít nhất hai lý do kết nối vẫn có thể lỗi dù rule này đúng: DB không lắng nghe, credential sai, DNS sai hoặc route thiếu. Bài tập muốn bạn tách **network reachability** khỏi **authentication**.

Từ trình duyệt đến tiến trình ứng dụng

Người dùng gõ tên miền. DNS trả địa chỉ; client tạo kết nối TCP; TLS xác thực máy chủ và mã hóa; HTTP request tới load balancer/reverse proxy; proxy định tuyến tới ứng dụng; ứng dụng gọi database/cache/queue rồi trả response. Với Kubernetes, đường đi có thể thêm cloud load balancer -> ingress controller/gateway -> Service -> Pod. Mỗi hop có log, timeout, giới hạn kết nối và health check khác nhau.

502 Bad Gateway thường cho thấy proxy không nhận được phản hồi hợp lệ từ upstream; **503** có thể do thiếu backend sẵn sàng; **504** là timeout ở một gateway. Đây là manh mối, không phải chẩn đoán cuối. Cần xem **proxy nào** trả mã, upstream status, request ID, thời gian, pod endpoints và log ứng dụng. Một request chậm 30 giây có thể bị timeout ở proxy dù ứng dụng vẫn xử lý xong và ghi database; retry từ client khi đó có nguy cơ tạo thao tác trùng.

Bài tập đọc luồng lỗi

Notes API local trả **/health** bình thường nhưng người dùng qua tên miền nhận 502. Liệt kê kiểm tra từ ngoài vào: DNS trỏ đúng chưa; TLS handshake; proxy có route đúng path/host; upstream host/port; service process có lắng nghe; firewall; application log. Với mỗi bước viết lệnh hoặc tín hiệu định dùng và kết quả nào sẽ loại trừ giả thuyết. Không bắt đầu bằng restart mọi thành phần: restart có thể làm mất bằng chứng.

Theo dấu một request, không học giao thức rời nhau

Khi bạn gõ **https://notes.example.com/notes**, trình duyệt cần phân giải tên, chọn IP, mở TCP tới cổng 443, thương lượng TLS, gửi HTTP request; reverse proxy có thể tiếp tục gọi upstream và upstream gọi DB. Mỗi chặng có kiểu lỗi riêng. **NXDOMAIN** xảy ra trước TCP. TCP timeout khác TLS certificate mismatch. HTTP 502 nghĩa một HTTP proxy đã trả lời, thường vì không nói chuyện được với upstream hợp lệ; HTTP 500 thường do server xử lý request nhưng lỗi nội bộ. Mã 502/503/504 phụ thuộc proxy cụ thể, vì vậy hãy đọc log của lớp phát mã thay vì coi bảng mã là chẩn đoán cuối.

Một IP không đủ xác định dịch vụ: cần giao thức, cổng, tên host và đôi khi đường URL. Ở TLS, Server Name Indication giúp server chọn chứng chỉ; ở HTTP, **Host** hoặc **:authority** giúp proxy chọn virtual host. Gọi thẳng IP có thể cho kết quả khác domain, dù cùng server. Nếu DNS chuyển domain sang IP mới mà bạn vẫn thấy IP cũ, phải xét TTL/cache ở resolver/client/proxy, không chỉ kiểm tra bản ghi authoritative.

Ví dụ bốn phép thử có ý nghĩa khác nhau

nslookup notes.example.com kiểm tra một góc của DNS nhưng không chứng minh cổng mở. **Test-NetConnection notes.example.com -Port 443** trên PowerShell hoặc **nc -vz host 443** trên Linux kiểm tra TCP connect, chưa chứng minh TLS/HTTP. **curl -vI https://notes.example.com/** cho thấy TLS và HTTP headers, nhưng **HEAD /** có thể không tương đương **POST /notes**. Request nghiệp vụ có ID và body cụ thể mới kiểm tra ứng dụng. Ghi kết quả **thực** ở mỗi lớp; đừng ghi “network OK” sau một ping, vì ICMP có thể bị chặn hoặc đi đường khác TCP 443.

CIDR và route qua một phép tính

Mạng **10.20.4.0/24** có 256 địa chỉ trong IPv4; trong thiết kế truyền thống, .0 là network và .255 là broadcast, nhưng số địa chỉ dùng được trên cloud còn phụ thuộc provider. Một subnet **10.20.4.0/24** và subnet **10.20.5.0/24** không chồng lấp; **10.20.4.128/25** nằm trong **/24** đầu. Nếu VNet/VPC on-prem và cloud dùng dải chồng lấp, route qua VPN có thể mơ hồ dù firewall mở. Trước khi tạo hybrid VPN, lập bảng CIDR của từng site, route table, NAT, DNS và owner. Một packet đi từ app đến DB cần route đi **và** route về; chỉ thấy SYN đi không chứng minh đường hồi đáp.

Security Group/NSG/firewall là lớp lọc; route xác định đường đi. Mở port trong firewall không tạo route. Một Private Endpoint mang IP riêng không tự bảo đảm hostname app phân giải về IP riêng; DNS zone/link và resolver path là phần của thiết kế. Chẩn đoán theo thứ tự: phân giải domain ở **nơi app chạy**, xem IP, route, TCP, TLS, auth. Tránh thử từ laptop rồi suy ra Pod trong cluster cũng đi cùng đường.

Lab local: quan sát DNS, TCP và HTTP

Điều kiện: PowerShell hoặc Linux terminal, Python 3 và curl; dùng máy cá nhân. Ở terminal A chạy **python -m http.server 8765 --bind 127.0.0.1** (Linux có thể dùng **python3**). Ở terminal B thử **curl -v http://127.0.0.1:8765/**. **Dự kiến:** TCP connect rồi HTTP response 200 nếu server chạy và thư mục cho đọc. Tiếp tục gọi **/khong-co-file** để nhận HTTP 404: TCP vẫn thành công, lỗi nằm ở tài nguyên HTTP. Dừng server rồi gọi lại: kết nối thường bị từ chối. Ghi rõ ba dấu hiệu khác nhau trong output.

Tạo một file **hello.txt** trong thư mục lab và thử **curl -v http://localhost:8765/hello.txt**. **localhost** có thể phân giải IPv4/IPv6; nếu client thử **::1** trước nhưng server chỉ bind IPv4, curl có thể thử tiếp **127.0.0.1**. Đọc từng dòng connect thay vì kết luận từ một địa chỉ đầu tiên. Trên Windows, **curl** có thể là alias ở PowerShell cũ; dùng **curl.exe** nếu cần để xem output curl thực. Không mở server ra **0.0.0.0** trên mạng công cộng cho bài này.

Bài tập lỗi proxy giả định: proxy nghe 443, upstream cấu hình **127.0.0.1:8766** nhưng app nghe 8765. Client có thể hoàn tất DNS/TCP/TLS tới proxy rồi nhận 502. Sửa upstream port và kiểm chứng bằng request qua proxy; đổi DNS không chữa upstream sai. Nếu proxy chạy trong container, **127.0.0.1** là loopback của **container proxy**, không phải host/app container; cần tên service/mạng đúng.

TLS và HTTP cho người vận hành

TLS cung cấp xác thực server qua certificate chain và mã hóa trên đoạn kết nối. Kiểm tra tên domain trong SAN, thời hạn, issuer/trust chain và thời gian hệ thống. Certificate hợp lệ trên client của bạn có thể không hợp lệ trong container thiếu CA phù hợp. TLS termination ở load balancer khiến đoạn LB->upstream là một kết nối khác; cần quyết định có mã hóa tiếp hay không, header **X-Forwarded-Proto**/client IP có được tin cậy từ proxy đúng không. Không tắt verify TLS trong script production để “cho chạy”.

HTTP là giao thức request/response: method, path, headers, body, status. GET nên không thay state theo kỳ vọng phổ biến; POST có thể tạo tài nguyên và cần chiến lược retry/idempotency. Timeout ở client có thể xảy ra sau khi server đã ghi DB nhưng trước khi response tới client. Nếu client retry POST không có idempotency key, dữ liệu có thể bị tạo đôi; đây là vấn đề ứng dụng và mạng cùng tham gia.

WebSocket bắt đầu bằng handshake HTTP rồi nâng cấp thành kết nối lâu dài; proxy timeout/upgrade headers ảnh hưởng khác request ngắn.

Ca điều tra có lời giải: API chỉ lỗi từ Kubernetes

Laptop gọi DB cloud được, Pod gọi báo timeout. Bước đầu không phải tăng timeout. Trong Pod, xác định hostname phân giải ra IP nào; so với laptop. Nếu Pod dùng Private Endpoint IP, kiểm tra route/NSG/network policy từ subnet node/Pod và firewall DB, rồi kiểm tra đường về. Nếu laptop dùng public IP, hai phép thử không tương đương. Nếu TCP kết nối được nhưng auth lỗi, chuyển sang identity/credential/TLS. Nếu chỉ một node lỗi, so route/CNI/DNS cache của node đó. Câu trả lời tốt nói rõ bằng chứng nào sẽ bác bỏ từng giả thuyết.

Bạn đạt bài khi vẽ request theo các lớp DNS -> route/TCP -> TLS -> HTTP -> upstream -> DB và chọn được phép thử cho từng lớp. Tài liệu chuẩn: RFC 9110 HTTP semantics, RFC 8446 TLS 1.3.

Phân biệt lỗi bằng thời điểm request dừng

Một client báo “không vào được web” là triệu chứng quá rộng. Dùng timeline: **phân giải tên** (DNS trả gì), **mở kết nối** (SYN/SYN-ACK hoặc timeout), **bắt tay TLS** (chứng chỉ/SNI), **HTTP** (status/headers), **proxy upstream** (log và thời gian), **app/DB**. Ghi timestamp và điểm đo của từng phép thử. Nếu **curl** từ laptop thành công nhưng Pod lỗi, phép thử chỉ nói đường laptop tốt; phải chạy từ đúng Pod/node/subnet và đúng DNS resolver. Nếu lỗi chỉ từ một khu vực, xem CDN/LB/DNS/route theo khu vực trước khi restart API toàn cầu.

Đọc timeout đúng lớp

DNS timeout: resolver không trả lời kịp, có thể do network tới DNS hoặc overload. TCP connect timeout: không hoàn tất handshake tới IP:port; có thể route, firewall, server không nhận/packet drop. TLS handshake timeout: TCP đã kết nối, nhưng handshake không hoàn tất; có thể proxy/CPU/certificate negotiation/network. HTTP read timeout: request có thể đã tới server, server chậm hoặc response bị kẹt; retry POST có thể tạo side effect lặp. Bốn timeout này cần metric/log khác nhau. Nếu thư viện chỉ báo “timeout” chung, cấu hình/log instrumentation để thấy phase.

Ví dụ tính tải kết nối

Một API nhận 500 request/giây; mỗi request giữ connection DB trung bình 200 ms. Theo phép tính xấp xỉ của Little's Law, trung bình khoảng 100 request đồng thời ở đoạn DB (**500 x 0,2**). Nếu pool DB chỉ 50, request phải chờ dù CPU app còn rảnh; nếu latency DB tăng lên 1 giây, nhu cầu đồng thời xấp xỉ 500. Đây là ví dụ lý tưởng hóa; traffic burst, keepalive, queue và timeout làm thực tế khác. Nó giải thích vì sao một lỗi DB chậm có thể làm pool cạn rồi kéo theo HTTP 504 ở proxy. Tăng pool lên 500 có thể đẩy overload sang DB; cần tối ưu query, backpressure và timeout budget.

Timeout budget end-to-end

Giả sử client timeout 3 giây, edge proxy timeout 5 giây, app gọi DB timeout 10 giây và retry hai lần. Client đã bỏ đi trước khi app dừng; app vẫn dùng tài nguyên, làm tải tăng. Thiết kế deadline lan truyền: tổng thời gian các bước + retry/backoff phải nằm trong ngân sách request, có chút dự phòng. Retry chỉ khi lỗi có khả năng tạm thời và operation idempotent hoặc có idempotency key. Nếu tất cả tầng đều retry 3 lần, một request có thể nhân tải rất lớn vào dependency đang lỗi.

Bài phỏng vấn có lời giải

DNS trả đúng IP, ping được, vì sao HTTPS vẫn lỗi? Ping ICMP không chứng minh TCP 443/TLS/HTTP; kiểm tra connect, SNI/cert, Host header và proxy. **Có HTTP 404, network đã tốt hoàn toàn chưa?** Đã có đường tới một HTTP server, nhưng có thể là sai virtual host/route/upstream; cần xác định server nào trả 404. **Có 502 từ Nginx, app có thấy request?** Có thể không; đọc access/error log Nginx và app log cùng request ID. **Private Endpoint DNS đúng nhưng TCP timeout?** Xem route, NSG/firewall và chiều hồi đáp; identity chưa được kiểm tra vì chưa tới bước auth.

Bạn đạt bài khi một đồng nghiệp đưa một triệu chứng mơ hồ, bạn đặt được năm câu hỏi phân lớp và chọn phép thử chỉ tác động read-only. Tham chiếu HTTP semantics và TLS 1.3 khi cần cơ chế chính xác.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://www.rfc-editor.org/rfc/rfc1035
- https://www.rfc-editor.org/rfc/rfc9110
- https://www.rfc-editor.org/rfc/rfc8446